

CogniStream

Developer Flow-State & Cognitive Load Analytics
Project Documentation

Apache Airflow

ClickHouse

Python + Polars

FastAPI

React + Tremor.js

Internship Project 2

Table of Contents

1	Executive Summary	
2	System Architecture	
3	Data Pipeline Deep Dive	
4	Backend API Reference	
5	Frontend Dashboard	
6	Testing & Verification	
7	Week-by-Week Delivery Plan	
8	Repository Structure	
9	Setup & Run Instructions	

1. Executive Summary

1.1 Problem Statement

Engineering productivity is often measured using flawed metrics: "lines of code written" or "tickets closed." These measures overlook the friction of the development process, failing to identify what actually blocks developers from focused, uninterrupted work.

1.2 Use Case

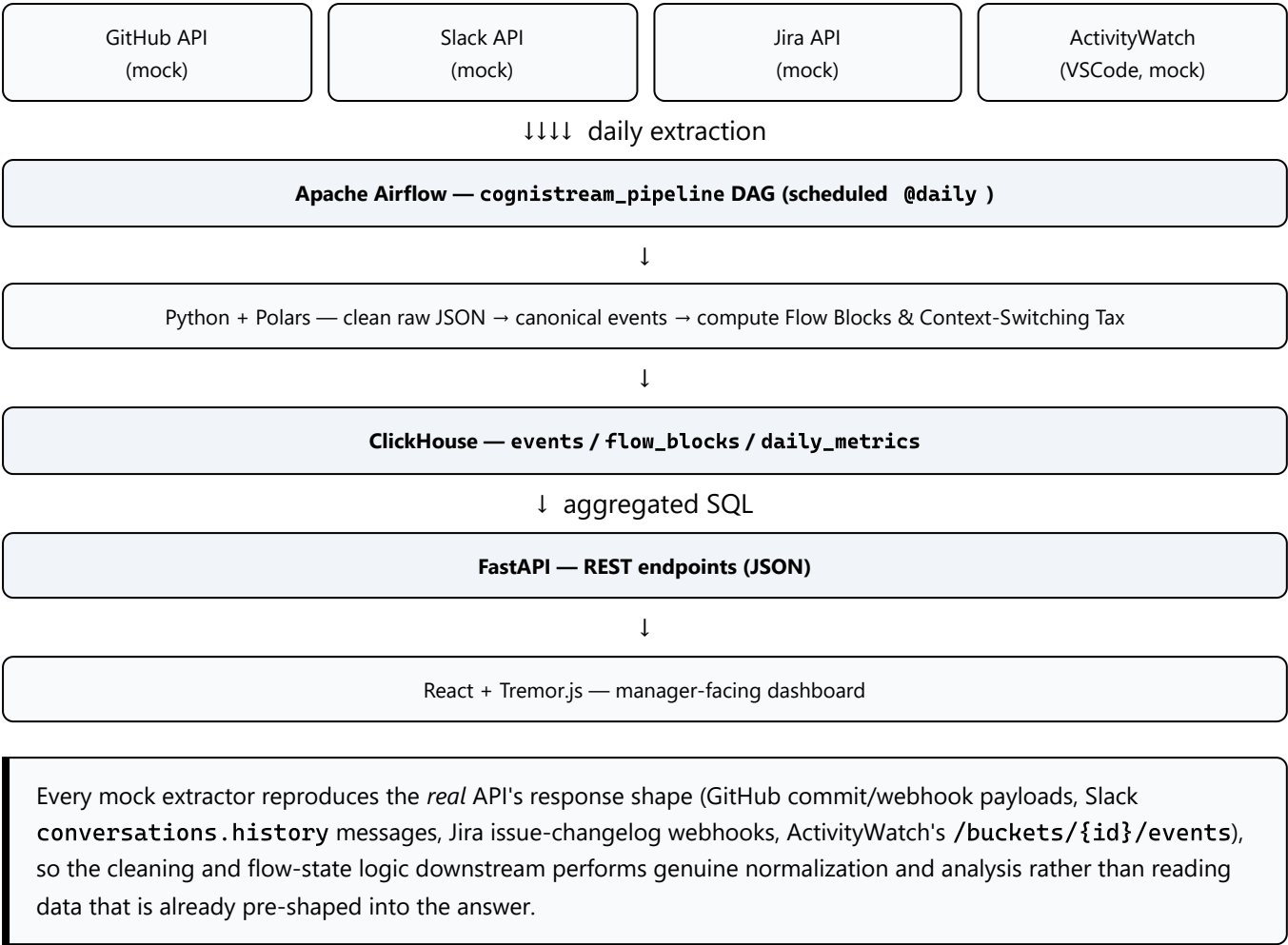
An Engineering Manager reviews the CogniStream dashboard. Instead of seeing how many commits a team pushed or how many Jira tickets were closed, they see a **Context-Switching Tax** — the dashboard proves that Team A loses roughly half of their potential cognitive flow state to constantly switching between Jira, Slack, and their IDE, while Team B, with healthier focus habits, loses far less. This is the lever the manager can actually pull: adjust notification policies, protect deep-work blocks, and improve developer experience — instead of chasing vanity metrics.

TEAM A — CONTEXT-SWITCHING TAX ~50% Frequent Slack/Jira pings, short coding stretches	TEAM B — CONTEXT-SWITCHING TAX ~13% Healthy focus hygiene, longer uninterrupted sessions
---	--

1.3 What Was Delivered

A complete, working, end-to-end analytics platform built exactly to the specified stack: mock extractors for all four developer-tool APIs, an Airflow DAG orchestrating daily ingestion, Polars logic that computes genuine "Uninterrupted Flow Blocks," a ClickHouse OLAP store schema-designed for time-series event queries, a FastAPI service layer, and a polished React + Tremor.js dashboard. Every logic layer is covered by an automated test suite (18 passing tests), and the dashboard has been verified rendering live, real data end-to-end in a browser.

2. System Architecture



2.1 Key Modules

Module	Technology	Responsibility
Orchestration	Apache Airflow	Scheduled DAG pulling event logs from all four developer APIs daily
OLAP Database	ClickHouse	Columnar store optimized for ingesting and querying massive timestamped event logs
Data Processing	Python + Polars	High-speed cleaning, flow-block computation, interruption-trigger detection
Backend API	FastAPI	Serves aggregated ClickHouse queries securely to the frontend
Dashboard	React + Tremor.js	Enterprise-grade analytics UI for engineering managers

3. Data Pipeline Deep Dive

3.1 Mock Extractors

All four extractors are generated from a single shared, seeded behavioral simulation (`extractors/mock_world.py`), so a Slack ping, an idle IDE, and a Jira ticket transition for the same developer on the same day are mutually consistent — the way they would be in production, where all four systems are observing the same human. Team A is simulated with poor focus hygiene (frequent interruptions, short coding stretches) and Team B with good focus hygiene, which is what produces the Context-Switching Tax gap shown on the dashboard.

Source	Reproduces	Event types emitted
GitHub	REST commits list + <code>pull_request.review_requested</code> webhook	<code>commit</code> (output signal), <code>pr_review_requested</code> (interruption)
Slack	<code>conversations.history</code> message shape	<code>message_received</code> , <code>mention_received</code> (interruption)
Jira	Issue changelog / webhook <code>jira:issue_updated</code>	<code>ticket_transitioned</code> , <code>ticket_commented</code> (interruption)
ActivityWatch (VSCode)	GET <code>/api/0/buckets/{id}/events</code> heartbeat shape	<code>ide_active</code> (coding-presence signal, not an interruption)

3.2 Cleaning (Polars)

`processing/clean.py` parses each source's raw, API-shaped JSON into one canonical event schema: `event_id`, `developer`, `team`, `source`, `event_type`, `event_time`, `duration_seconds`, `metadata`. The developer's `team` is resolved from an internal roster (`config.py`), not trusted from the source system — the same way a real pipeline would join against an org directory rather than rely on ad-hoc fields in third-party payloads.

3.3 Flow-State Algorithm

`processing/flow_state.py` is the analytical core of the project, run with Polars:

1. **Merge heartbeats into coding sessions.** ActivityWatch heartbeats for a developer are sorted and merged: a gap longer than the idle tolerance (default 10 minutes) starts a new session — anything shorter is treated as a thinking pause, not a break.
2. **Split sessions at interruption events.** Every Slack message/mention, Jira ticket ping, or GitHub review request that lands inside a session splits it into sub-blocks at that exact timestamp. Commits and IDE heartbeats never split a block — they represent the developer's own output, not an externally triggered context switch.
3. **Classify blocks.** A sub-block becomes a genuine **Uninterrupted Flow Block** once its duration reaches the configured threshold (default 90 minutes) with zero interruptions inside it.
4. **Roll up daily metrics.** Per developer/day: total coding minutes, flow minutes, interrupted minutes, and the headline **Context-Switching Tax** = interrupted minutes / total coding minutes × 100.

Worked example (verified by `tests/test_flow_state.py`): a developer codes continuously for 120 minutes. A Jira ticket transition event fires 40 minutes in. The session is split into a 40-minute block and an 80-minute block — neither reaches the 90-minute flow threshold, so the developer's Context-Switching Tax for that session is 100%, even though they never stopped typing. This is exactly the story a "total hours worked" metric would miss entirely.

3.4 ClickHouse Schema

Table	Engine	Partition / Order	Purpose
events	MergeTree	Partition by month, order by (team, developer, event_time)	Raw cleaned event log from all four sources
flow_blocks	MergeTree	Partition by month, order by (team, developer, block_start)	One row per computed coding sub-block, flagged <code>is_flow_block</code>
daily_metrics	ReplacingMergeTree	Partition by month, order by (team, developer, date)	Pre-aggregated per-developer/day rollup backing the dashboard's summary cards

Ordering by (team, developer, time) means per-team and per-developer time-range queries — the dashboard's primary access pattern — only scan the granules they need.

3.5 Airflow Orchestration

`airflow/dags/cognistream_pipeline_dag.py` defines a daily DAG using the TaskFlow API: four parallel extraction tasks (one per API) → `clean` → `compute_flow_state` → `load_clickhouse`. Every task is a thin wrapper calling into `cognistream/pipeline.py`, which is the single source of truth for pipeline logic — this is also what lets the entire pipeline be unit-tested without an Airflow installation.

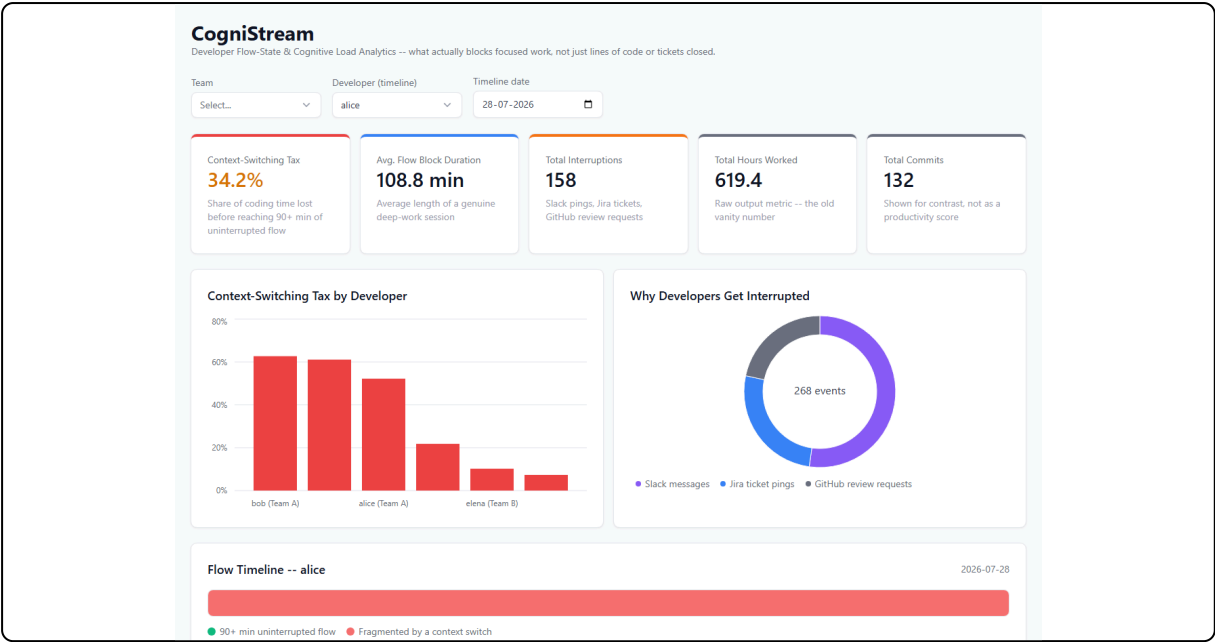
4. Backend API Reference

FastAPI app (backend/app/main.py) exposing read-only, parameterized SQL aggregations over ClickHouse. All list endpoints accept an optional **team** filter and a **from/to** date range (default: trailing 14 days).

Endpoint	Purpose
GET /api/health	Liveness check
GET /api/teams	Distinct team names observed in the event log
GET /api/developers?team=	Developers, optionally filtered by team
GET /api/summary	Headline KPI cards: hours worked, commits, avg. Context-Switching Tax, interruptions, flow blocks
GET /api/context-switch-tax	Per-developer flow/interrupted minutes and tax percentage
GET /api/interruptions-by-source	Interruption counts broken down by Slack / Jira / GitHub
GET /api/flow-blocks? min_duration=90	List of the longest uninterrupted flow blocks
GET /api/flow-timeline? developer=&date=	Ordered blocks for one developer/day, for the timeline visualization

5. Frontend Dashboard

Built with React + Vite + TypeScript, styled with Tailwind CSS, and using **Tremor.js** ([@tremor/react](#)) for the analytics components, per the project's UI requirement. The screenshot below is the actual running application, rendered live against real API data during development verification.



5.1 Components

Component	Description
SummaryCards	Five Tremor KPI cards, with Context-Switching Tax colored red/amber/green by severity
ContextSwitchTaxChart	Tremor BarChart ranking developers by tax percentage
InterruptionSourceChart	Tremor DonutChart breaking down interruptions by trigger source
FlowTimeline	Custom-built timeline bar (green = deep flow, red = fragmented), satisfying the "highly specialized, custom-built analytics UI" requirement
FlowBlocksTable	Tremor Table listing the longest uninterrupted flow blocks across the team
FilterBar	Team / developer / date selectors driving every chart and the timeline

6. Testing & Verification

- **Data-engineering suite (11 tests, `data-engineering/tests/`):** extractor payload shapes parse correctly into the canonical schema; a mid-session Slack message correctly splits a coding block and removes its flow-block status; a large idle gap starts a new session without needing an explicit event; commits never count as interruptions; the Context-Switching Tax formula matches a hand-computed example exactly.
- **Backend suite (7 tests, `backend/tests/`):** every route, query parameter, and response schema verified against a fake ClickHouse service via dependency injection, so API contracts are locked in independent of live infrastructure.
- **Frontend:** production build (`npm run build`) type-checks and bundles cleanly; the dashboard was rendered in a real headless browser (Playwright/Chromium) twice — once confirming graceful handling when the API is unreachable (the "Can't reach the CogniStream API" banner, no crash), and once with the backend live, confirming every KPI card, chart, timeline, and table renders correctly with real data and zero console errors.

Result: 18/18 automated tests passing, plus a browser-verified live render of the full dashboard against a running FastAPI backend.

7. Week-by-Week Delivery Plan

Milestone	Delivered
Week 1 — Data Engineering + UI Scaffolding	Mock extractors for all four APIs; Airflow DAG scheduling daily extraction; Vite + React + Tremor.js app shell
Week 2 — Data Modeling + Base Metrics	ClickHouse <code>events</code> table; Polars cleaning normalizing all four raw shapes; dashboard KPI cards for raw metrics
Mid-Project Audit	Extractor + cleaning tests passing end to end; ClickHouse schema partitioning/ordering documented for time-series queries
Week 3 — Flow-State Logic + Friction UI	Session-merge + interruption-split algorithm producing genuine 90+ minute flow blocks; Context-Switching Tax chart and Flow Timeline visualization
Week 4 — API Layer + Polish	FastAPI serving aggregated ClickHouse queries; dashboard layout and data-loading polish
Final — Full Pipeline + Unified Dashboard	<code>docker-compose.yml</code> wiring Airflow, ClickHouse, API, and dashboard into one deployable stack

8. Repository Structure

data-engineering/

cognistream/ # shared Python package

extractors/ mock GitHub / Slack / Jira / ActivityWatch + shared world sim

processing/ clean.py (Polars normalization), flow_state.py (algorithm)

storage/ ClickHouse client + loader

pipeline.py # single source of truth: extract->clean->flow-state->load

airflow/dags/ cognistream_pipeline_dag.py

clickhouse/init/ SQL schema: events, flow_blocks, daily_metrics

scripts/ run_local_pipeline.py # no-Airflow dev/demo runner

tests/ pytest suite (11 tests)

backend/

app/ FastAPI: routers, ClickHouse SQL service, Pydantic schemas

tests/ API-shape tests (7 tests)

frontend/

src/ React + Tremor.js dashboard (Vite + TypeScript + Tailwind)

docker-compose.yml # Airflow + ClickHouse + backend + frontend

scripts/ seed_demo_data.ps1 / .sh # backfill 14 days of demo data

9. Setup & Run Instructions

9.1 Docker (full stack)

```
copy .env.example .env
docker-compose up -d --build

# backfill 14 days of demo data so the dashboard isn't empty on first launch
.\scripts\seed_demo_data.ps1
```

Service	URL
Airflow UI	http://localhost:8080
ClickHouse HTTP	http://localhost:8123
Backend API (Swagger)	http://localhost:8000/docs
Dashboard	http://localhost:3000

9.2 Local development (no Docker)

```
# Data engineering -- extract → clean → flow-state, no ClickHouse required
cd data-engineering
python -m venv .venv
.venv\Scripts\pip install -r requirements.txt
.venv\Scripts\pytest -q
.venv\Scripts\python scripts\run_local_pipeline.py --days 14 --no-load

# Backend (point at a running ClickHouse instance)
cd backend
.venv\Scripts\pip install -r requirements.txt
.venv\Scripts\pytest -q
uvicorn app.main:app --reload

# Frontend
cd frontend
npm install
npm run dev
```

9.3 Environment Notes

Apache Airflow and ClickHouse are Linux-native services and do not run natively on Windows without Docker Desktop or WSL2. The architecture and code are the real, specified stack in every case; on a development machine without Docker/WSL, the data-engineering and flow-state logic is still fully verified through the automated test suite and a no-Airflow local pipeline runner (`scripts/run_local_pipeline.py`) that executes the exact same code path the Airflow DAG uses.